



HỌC VIỆN NGÂN HÀNG
KHOA CÔNG NGHỆ THÔNG TIN VÀ KINH TẾ SỐ

CHƯƠNG 4
PHƯƠNG PHÁP KIẾN TRÚC
PHẦN MỀM

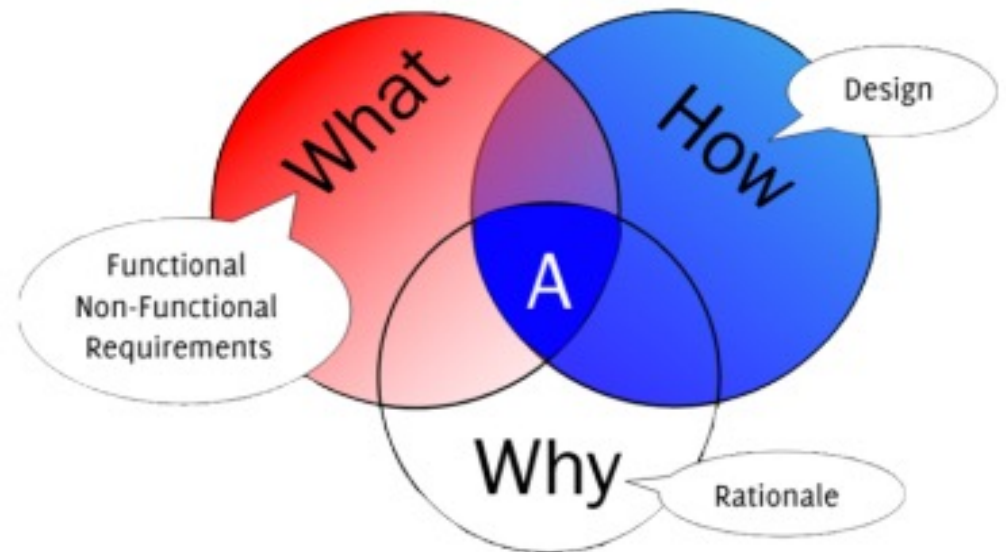
Hà Nội - 2024

Nội dung trình bày

- ❖ Một số khái niệm
- ❖ Một số Phương pháp Kiến trúc phần mềm

Phương pháp Kiến trúc phần mềm

- ❖ Phương pháp kiến trúc phần mềm (**Software Architecture Methodology**) là một cách tiếp cận hệ thống hóa và tổ chức các thành phần của một hệ thống phần mềm. Nó không chỉ tập trung vào việc thiết kế các thành phần cụ thể, mà còn quan tâm đến cách các thành phần này tương tác với nhau, cách chúng được triển khai và mở rộng trong thời gian.



Một số Phương pháp Kiến trúc phần mềm (1)

- ❖ **Kiến trúc phân lớp (Layered Architecture):** Hệ thống được phân chia thành các lớp hoạt động ở mức độ trừu tượng khác nhau. Các lớp này thường được xác định dựa trên chức năng hoặc mức độ trừu tượng của nhiệm vụ.
- ❖ **Kiến trúc Hướng thành phần (Component-Based Architecture):** Hệ thống được xây dựng từ các thành phần độc lập, có thể tái sử dụng. Các thành phần này thường được kết hợp lại để tạo thành các ứng dụng lớn hơn.
- ❖ **Kiến trúc Microservices:** Một biến thể của kiến trúc dựa trên dịch vụ, trong đó các ứng dụng được chia thành các dịch vụ nhỏ, độc lập và có thể mở rộng riêng biệt. Điều này tăng tính linh hoạt và khả năng mở rộng của hệ thống.

Một số Phương pháp Kiến trúc phần mềm (2)

- ❖ **Kiến trúc Hướng dịch vụ (Service-Oriented Architecture - SOA):** Đây là một kiểu kiến trúc trong đó các ứng dụng được chia thành các dịch vụ độc lập, có thể tái sử dụng và tương tác thông qua các giao thức tiêu chuẩn như HTTP và SOAP.
- ❖ **Kiến trúc Lõi và Tiên tiến (Core and Satellite Architecture):** Mô hình này tập trung vào việc phân chia hệ thống thành hai phần: một lõi chính có thể mở rộng và các phần tiên tiến xung quanh nó, giúp giảm thiểu sự phức tạp và tăng tính linh hoạt.
- ❖ **Kiến trúc Dựa trên Sự kiện (Event-Driven Architecture - EDA):** Ở đây, các thành phần của hệ thống phản hồi và tương tác với nhau thông qua các sự kiện, thường là bất đồng bộ và lõi tín hiệu của hệ thống.

Kiến trúc phân lớp (Layered Architecture)

Khái niệm

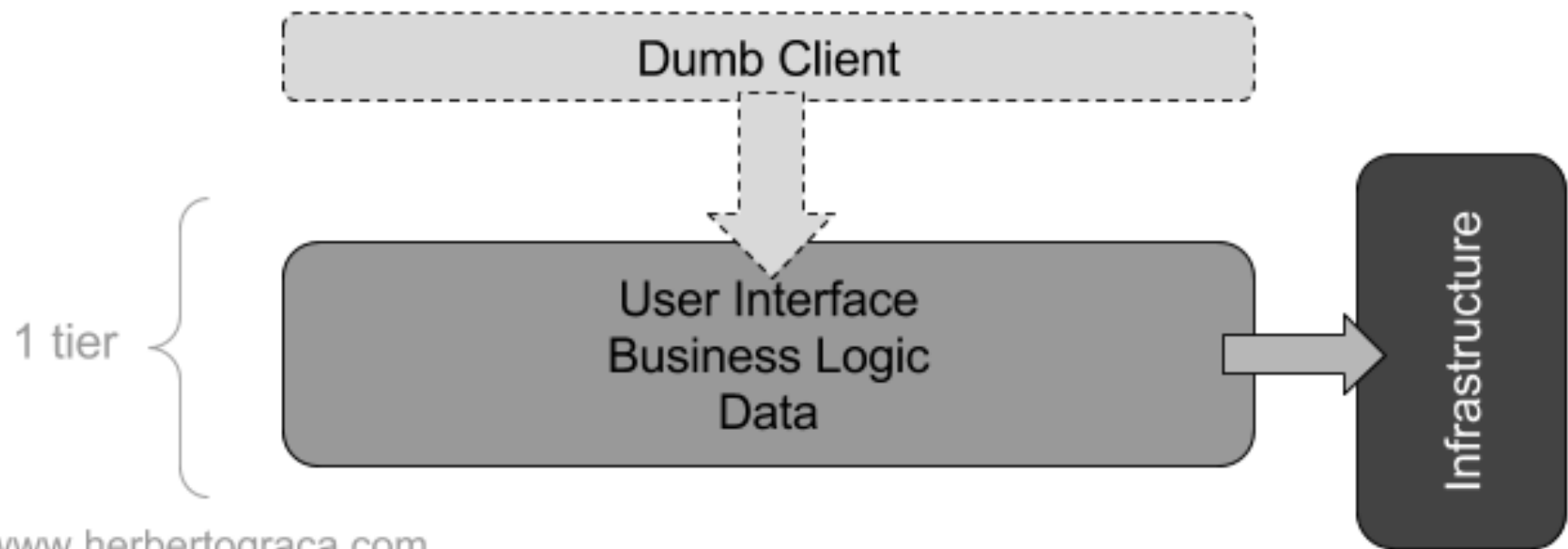
- ❖ Kiến trúc phân lớp dạng kiến trúc rất phổ biến ngày nay (kiến trúc 3 lớp, MVC, MVVM,...). Tất cả đều có chung một mục đích là tổ chức, phân rã mã nguồn theo nhiều chức năng và nhiệm vụ khác nhau trong hệ thống.

Phân lớp là gì?

- ❖ Trong một hệ thống phân lớp, thì một lớp:
 - Phụ thuộc vào các lớp bên dưới nó;
 - Không biết và không phụ thuộc vào các lớp bên trên sử dụng nó;
- ❖ Trong kiến trúc phân lớp, các layer có thể được design theo hướng nghiêm ngặt, một layer chỉ có hiểu biết và sử dụng layer ngay bên dưới nó, hoặc theo hướng linh hoạt hơn, nghĩa là một layer có thể sử dụng tất cả các layer ở dưới nó.
- ❖ Đôi khi các layer được thiết kế sao cho domain layer hoàn toàn không muốn presentation layer biết đến data source layer. Tuy nhiên, và thường xuyên, presentation layer nên truy cập trực tiếp đến data source layer. Mặc dù điều này không tinh khiết, nhưng nó có xu hướng hoạt động tốt hơn trong thực tế.

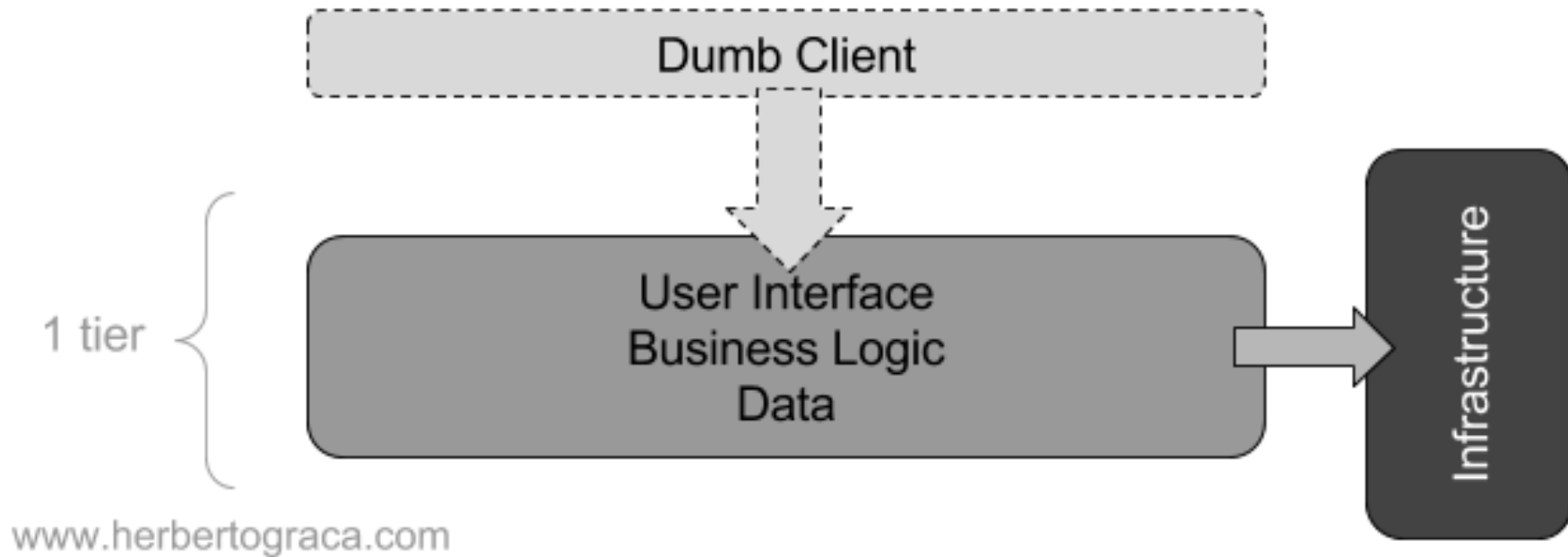
Kiến trúc phân lớp giai đoạn 60-70

- ❖ Những application lúc đó (hầu như) hoàn toàn khác với ngày nay. Chẳng có GUI (cái chỉ mới xuất hiện vào những năm đầu thập niên 90), tất cả application được sử dụng thông qua CLI (Command-line interface) dưới dạng cửa sổ console với nhiệm vụ đơn giản là nhận input (bàn phím) từ người dùng và xuất ngược trở ra màn hình. Cũng chẳng có khái niệm Client-Server, tất cả được đóng gói thành một bản cài đặt trên từng máy trạm.



Kiến trúc phân lớp giai đoạn 60-70

- ❖ Vì ứng dụng lúc đó quá đơn giản nên chẳng ai buồn nghĩ tới chuyện phân tách layer/tier này nọ làm gì. Ta có thể gọi các ứng dụng lúc đó với kiến trúc 1 layer, 1 tier, chúng không thể mở rộng (scalable), ví dụ, nếu chúng ta muốn cập nhật phiên bản mới, thì chỉ có cách là cài đặt chúng trên từng máy trạm (client).

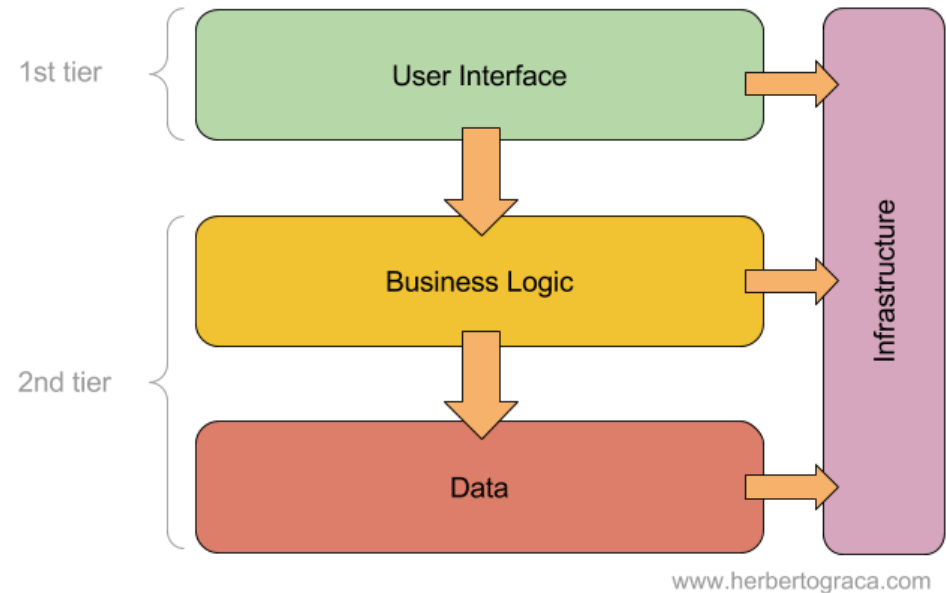


Kiến trúc phân lớp giai đoạn 80-90

- ❖ Giai đoạn này đã bắt đầu xuất hiện các ứng dụng dành cho doanh nghiệp (enterprise) với lượng người sử dụng dùng máy tính cá nhân (desktop computer) để truy cập vào ứng dụng thông qua network.
- ❖ Có 3 lớp được chia ra:
 - User Interface (Presentation): là những chương trình chạy trên desktop / CLI hoặc các web page. Các chương trình client lúc này thường là các rich client (nơi luồng công việc và kiểm tra input đầu vào đặt ở client)
 - Business Logic (Domain): là nơi đặt các logic về nghiệp vụ chính của hệ thống, layer này sẽ tiếp nhận các request từ phía Client, xử lý và lưu trữ data thông qua Data source layer.
 - Data source: nhiệm vụ chính của layer này là thực thi các tác vụ lưu trữ dữ liệu và liên lạc với các applications khác (được request từ business logic layer).

Kiến trúc phân lớp giai đoạn 80-90

- ❖ Đây chính là mô hình 3 lớp (3-layers) kinh điển. Tuy nhiên xét về mặt các lớp vật lý (tier) thì ứng dụng thường được tổ chức ở mức 2 tier hay còn gọi là Client-Server. Client side sẽ chứa Presentation layer, Server side sẽ chứa Business logic layer and Data source layer.
- ❖ Kiến trúc này có thể giải quyết các vấn đề về mở rộng, tuy nhiên vẫn còn hạn chế. Các ứng dụng dạng rich-client khi được cài đặt ở một số lượng lớn máy trạm thì việc update, mở rộng sẽ gặp nhiều khó khăn.

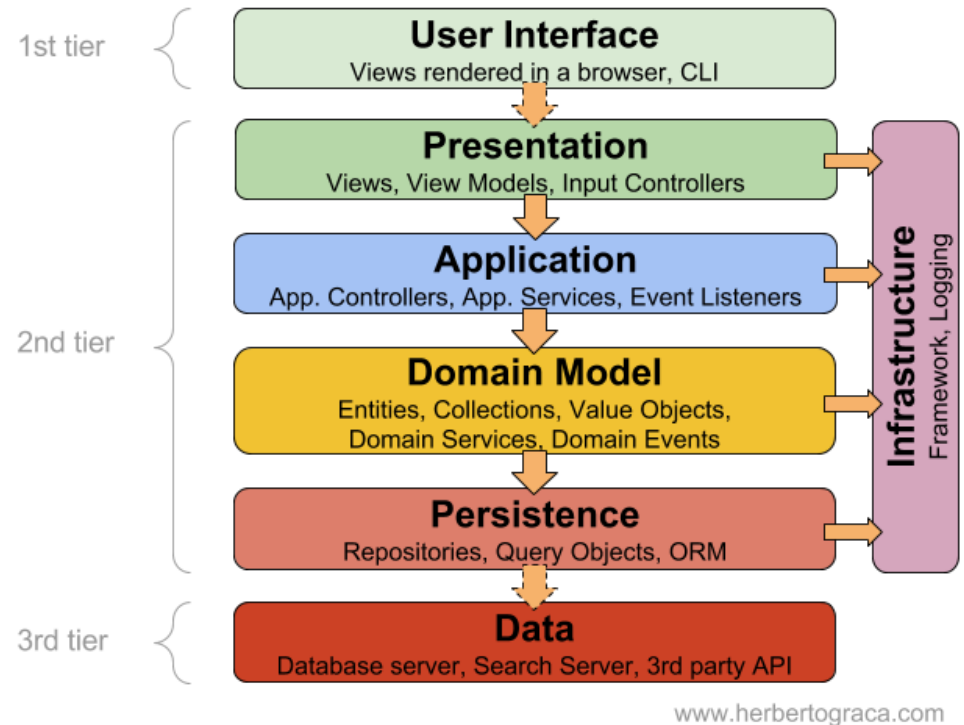


Kiến trúc phân lớp giữa những năm 90

- ❖ Đây là giai đoạn thế giới công nghệ có những chuyển biến mạnh mẽ. từ những năm 95 đến 2005, thế giới bắt đầu dịch chuyển lên “mây”. Số lượng người dùng tăng lên, độ phức tạp về ứng dụng tăng lên, vấn đề về cơ sở hạ tầng cũng ngày càng phức tạp.
- ❖ Từ đó dẫn đến việc cải tiến các mô hình kiến trúc sao cho phù hợp với nhu cầu, kiến trúc phân lớp vào thời điểm này được biến đổi như sau:
 - **Web browser:** làm nhiệm vụ render và gửi / nhận request tới server.
 - **Application server:** chứa presentation layer, the application layer, the domain layer, và the persistence layer.
 - **Database server:** được tối ưu cho mục đích lưu trữ dữ liệu

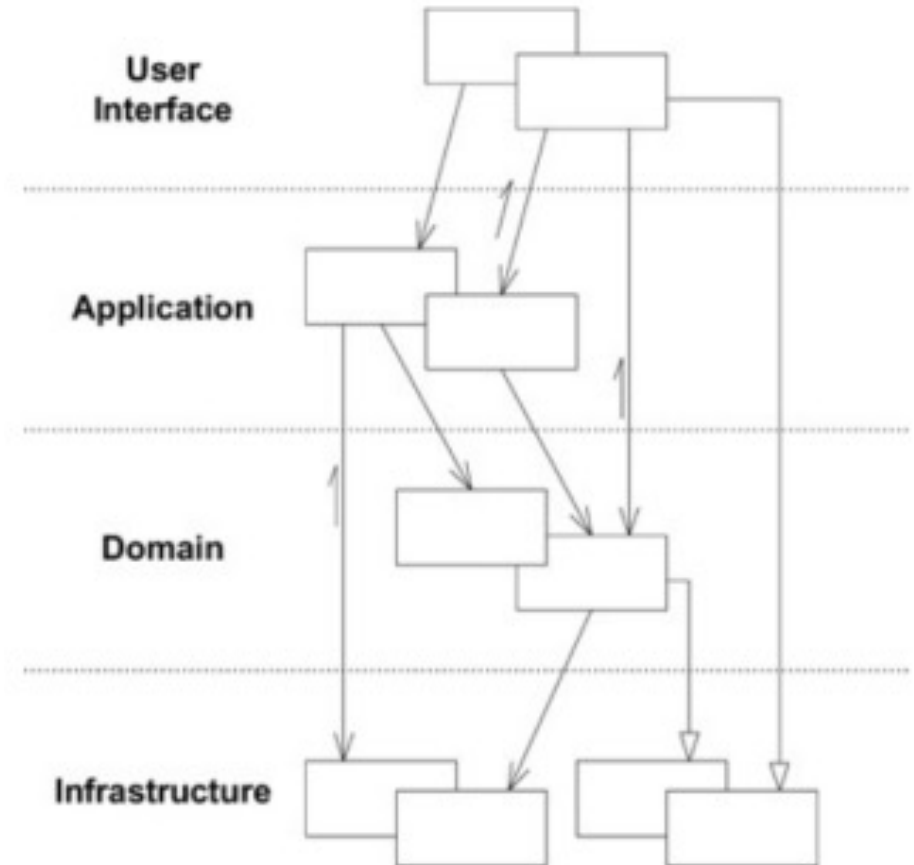
Kiến trúc phân lớp giữa những năm 90

- ❖ Đây là một architecture pattern 3 lớp vật lý (3-tier) hay n-tier. Tất cả logic đều được move lên phía server, do đó giải quyết triệt để vấn đề về mở rộng, client bây giờ chỉ làm nhiệm vụ render mà không cần biết các thay đổi từ server ra sao.



Kiến trúc phân lớp từ những năm 2000 - nay

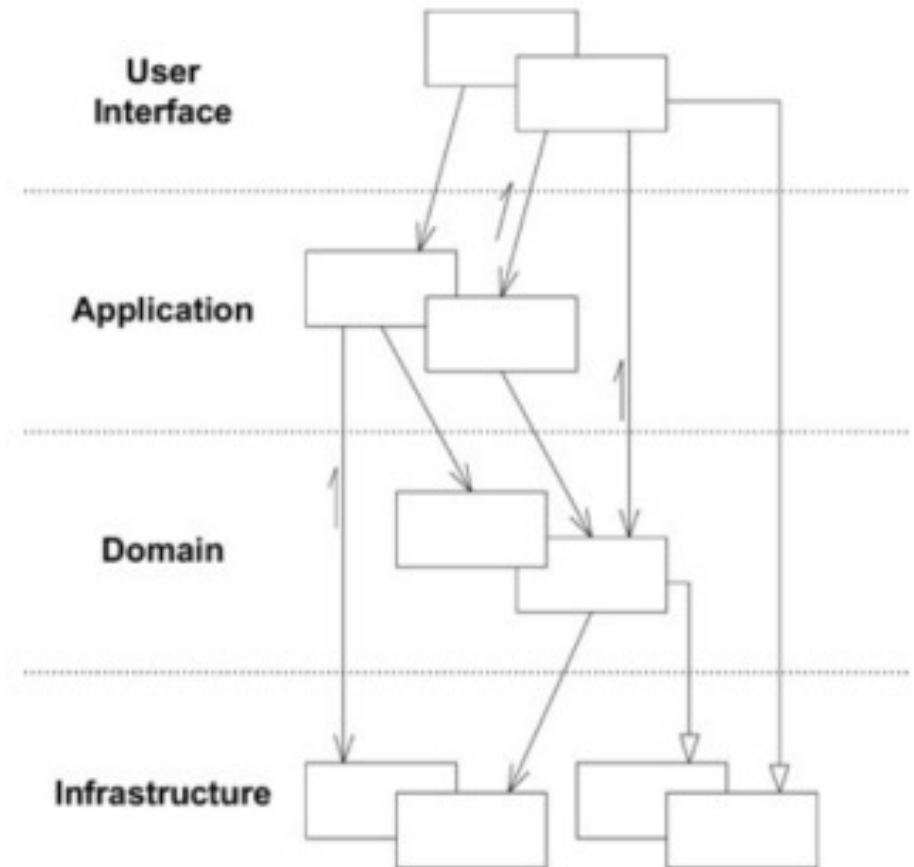
- ❖ Domain Driven Design là từ khóa rất hot trong giới kỹ nghệ lập trình. Eric Evan là người khởi xướng thuật ngữ này vào năm 2003 với cuốn sách nhan đề: Domain-Driven Design: Tackling Complexity in the Heart of Software. Cuốn sách chứa đựng rất nhiều khái niệm ảnh hưởng sâu rộng đến cách chúng ta định hình nên các bản thiết kế hiện nay.



Kiến trúc phân lớp từ những năm 2000 - nay

❖ User Interface

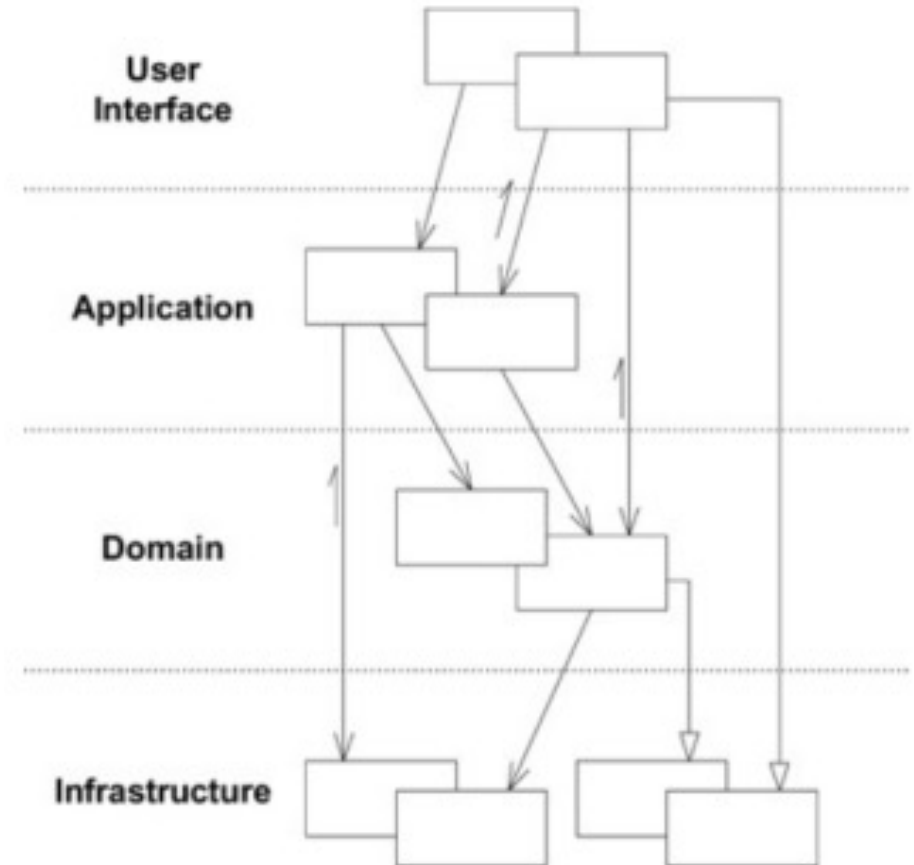
- Chịu trách nhiệm trong việc render và truyền tải thông điệp từ client tới server.
- Client ở đây có thể là người dùng hoặc những application khác.



Kiến trúc phân lớp từ những năm 2000 - nay

❖ Application Layer

- Điều khiển luồng công việc (use-case) bằng cách thực thi và kết hợp (Orchestrates) các Domain object. Nó không chứa business logic..
- Có 1 từ rất hay đó là **Orchestration**, nó được dùng trong SOA/ESB như là một điều phối viên.
- Ví dụ, Application Layer nhận 1 request từ Presentation, nó phải gọi xuống Domain Layer để thực hiện công việc đó, nhưng không đơn giản là 1 lời gọi đơn giản xuống Domain Layer, mà nó cần phải biết gọi Domain object nào, bao nhiêu Domain object để có thể thực hiện xong công việc, và trả về dữ liệu cho Presentation → người ta thường gọi nó là Application Services và Domain Services Layer.



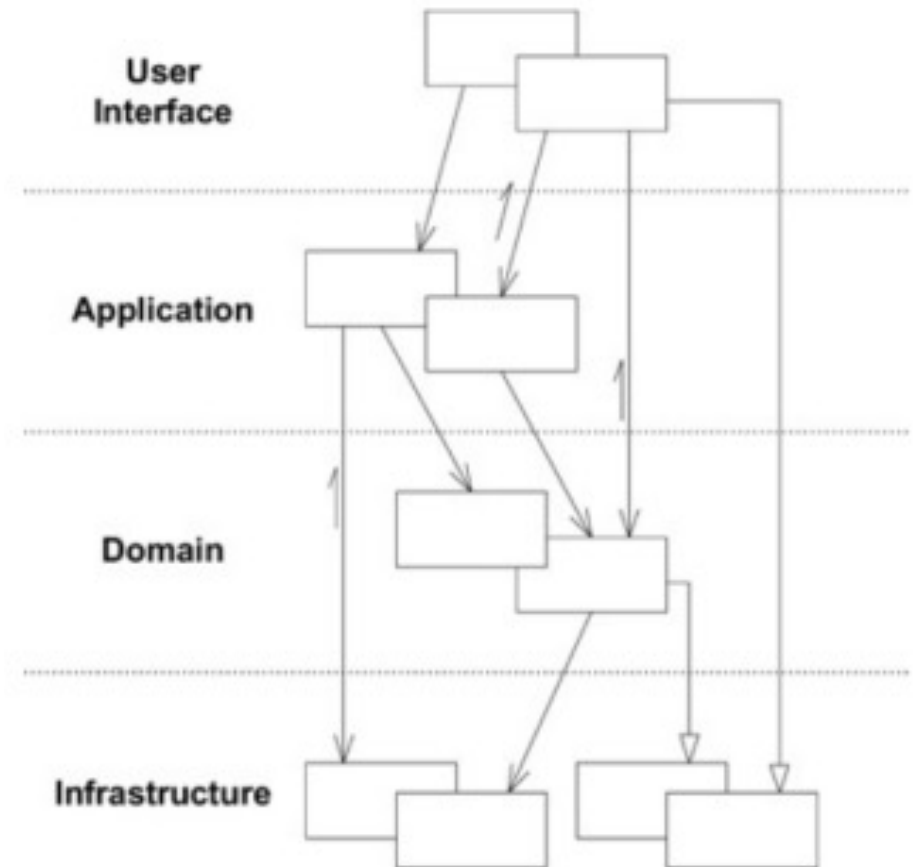
Kiến trúc phân lớp từ những năm 2000 - nay

❖ Domain Layer

- Layer này chứa các logic liên quan đến nghiệp vụ của ứng dụng, các Entities, Events, và bất cứ object nào handle logic nghiệp vụ. Rõ ràng, nó giống như **Entiy** trong **EBI Architecture**; là **trái tim** của cả hệ thống.

❖ Infrastructure

- Các common technical facilities dùng chung cho cả hệ thống, ví dụ như logging, persistance, messaging,...



ANTI-PATTERN: LASAGNA ARCHITECTURE

- ❖ Chúng ta quyết định đề ra các chính sách nghiêm ngặt về việc liên lạc giữa các layer (một layer chỉ có thể liên lạc với layer ở dưới ngay nó). Trong trường hợp như vậy, rất có thể ta phải tạo ra rất nhiều proxy method, proxy class trong các layer trung gian cốt chỉ để by-pass việc communication này, gây tốn công và khó chịu khi làm việc. Vì vậy, hãy giảm bớt sự nghiêm ngặt để có thể trực tiếp đi thẳng tới các layer phía dưới nó mà không cần qua trung gian.
- ❖ Một thay đổi nhỏ cũng có thể gây ảnh hưởng đến cả hệ thống. Ví dụ việc refactor lại 1 layer nào đó có thể gây ra hậu quả to lớn so với lợi ích nhận lại.
- ❖ Tạo quá nhiều layer dẫn đến việc phức tạp cho cả hệ thống.
- ❖ Có quá nhiều tier sẽ dẫn đến sự phức tạp và giảm performance trong hệ thống.

ANTI-PATTERN: LASAGNA ARCHITECTURE

- ❖ Việc tổ chức, đóng gói layer theo chức năng kỹ thuật (functional) của nó, ví dụ như UI, Domain, DB,.. mà không chia theo chức năng nghiệp vụ (component) như Product, Payment, Checkout, làm mất đi khả năng module hóa và đóng gói (encapsulation) theo domain concept.

Ưu điểm của Phân lớp

- ❖ Chúng ta chỉ cần hiểu những lớp bên dưới lớp chúng ta đang làm;
- ❖ Mỗi lớp có thể được thay thế bởi một thể hiện tương đương (equivalent implementation), không ảnh hưởng đến các lớp khác;
- ❖ Một lớp có thể được sử dụng bởi một số lớp cấp cao khác nhau

Nhược điểm của Phân lớp

- ❖ Lớp không thể đóng gói (encapsulate) tất cả mọi thứ (một field được thêm vào UI, rất có thể cũng cần phải được thêm vào DB);
- ❖ Các lớp bổ sung (extra) có thể gây ảnh hưởng đến hiệu suất, đặc biệt nếu ở các tier khác nhau.

Kết luận

- ❖ Layered Architecture là một cách chia để trị mỗi quan tâm, đóng gói và tách riêng, bằng cách nhóm các đơn vị mã bằng vai trò chức năng của chúng trong ứng dụng.
- ❖ Tuy nhiên, như hầu hết mọi thứ trong cuộc sống, quá nhiều sẽ phản tác dụng! Vì vậy, quy tắc của ngón tay cái (rule-of-thumb) là: Chỉ sử dụng các lớp chúng ta cần, các lớp chúng ta cần, và không có gì nhiều hơn nữa! Chúng ta không được đuổi theo đuổi một chén thánh kiến trúc, cái không hề tồn tại. Những gì tồn tại là một nhu cầu, và sự phù hợp tốt nhất có thể cho nhu cầu đó. Đó là một phần của Lean, tiện thể.
- ❖ Hơn nữa, điều quan trọng cần lưu ý là phương pháp tiếp cận top/down này đã lỗi thời. Không còn là những gì chúng ta nên làm trong phát triển phần mềm hiện đại.

Kiến trúc hướng dịch vụ (Service-Oriented Architecture - SOA)

Thực trạng

- ❖ Phần mềm ngày nay đang ngày càng trở nên phức tạp và dường như đang vượt khỏi khả năng kiểm soát của các mô hình phát triển phần mềm hiện có
- ❖ Nguyên nhân:
 - sự xuất hiện của nhiều công nghệ mới tạo nên môi trường không đồng nhất, trong khi nhu cầu về trao đổi, chia sẻ, tương tác giữa các hệ thống không thể đáp ứng được trong một môi trường như vậy
 - vấn đề lập trình dư thừa và không thể tái sử dụng

Giải pháp

- ❖ Có một cách tiếp cận giải quyết khá toàn diện mọi khó khăn nêu trên và nó đã được triển khai trong thực tế.
- ❖ Cách tiếp cận đó gọi là “Kiến trúc hướng dịch vụ” - Service-oriented Architecture (SOA).

Kiến trúc hướng dịch vụ là gì?

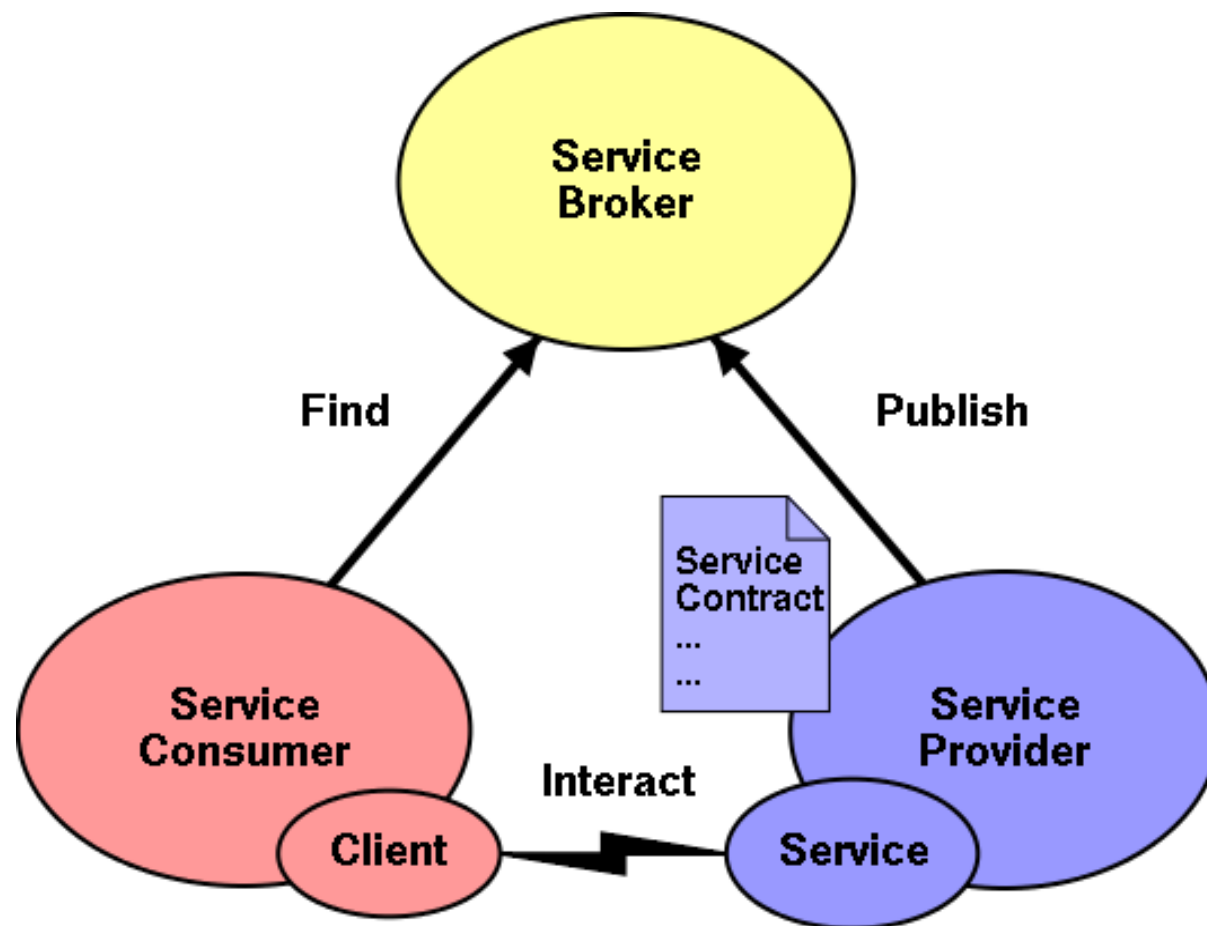
- ❖ Là một hướng tiếp cận với việc thiết kế và tích hợp các phần mềm, chức năng, hệ thống theo dạng module, trong đó mỗi module đóng vai trò là một “dịch vụ có tính loose coupling”, và có khả năng truy cập thông qua môi trường mạng.
- ❖ Là một tập hợp các dịch vụ được chuẩn hoá trên mạng trao đổi với nhau trong ngữ cảnh một tiến trình nghiệp vụ
- ❖ SOA giải quyết các vấn đề tồn tại của các hệ thống hiện nay như: phức tạp, không linh hoạt và không ổn định

Kiến trúc hướng dịch vụ là gì?

- ❖ SOA là một hướng tiếp cận với việc thiết kế và tích hợp các phần mềm hay chức năng hệ thống theo dạng module, trong đó mỗi module đóng vai trò là một “**dịch vụ** có tính kết nối lỏng lẻo” và có khả năng truy cập thông qua môi trường mạng.
- ❖ SOA là hướng tiếp cận xây dựng phần mềm dựa trên sự kết hợp các dịch vụ



Kiến trúc hướng dịch vụ là gì?



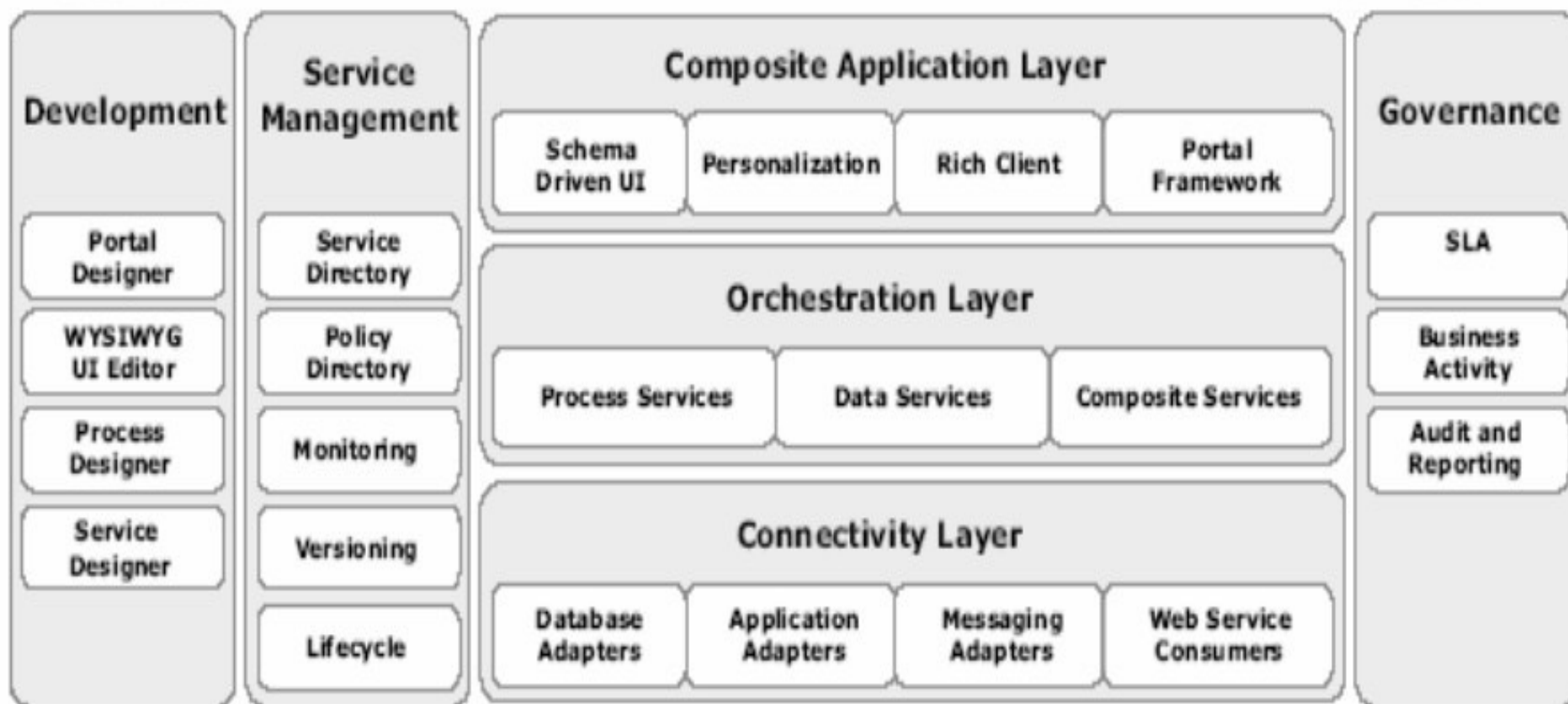
Các thành phần Kiến trúc hướng dịch vụ (1)

- ❖ **Dịch vụ (Service):** Là thành phần cơ bản của SOA, một dịch vụ là một phần của ứng dụng thực hiện một chức năng cụ thể và có khả năng tái sử dụng. Dịch vụ thường được truy cập thông qua các giao diện tiêu chuẩn như HTTP, SOAP hoặc REST.
- ❖ **Giao diện Dịch vụ (Service Interface):** Là cách mà các dịch vụ được truy cập và tương tác bên ngoài. Giao diện này định nghĩa các phương thức và tham số cần thiết để gọi và sử dụng dịch vụ.
- ❖ **Hợp đồng Dịch vụ (Service Contract):** Là một tài liệu hoặc một bản khai báo mà xác định các yêu cầu và cam kết giữa người sử dụng dịch vụ và nhà cung cấp dịch vụ. Hợp đồng này bao gồm các quy định về cách giao tiếp, định dạng dữ liệu và các điều kiện kỹ thuật khác.

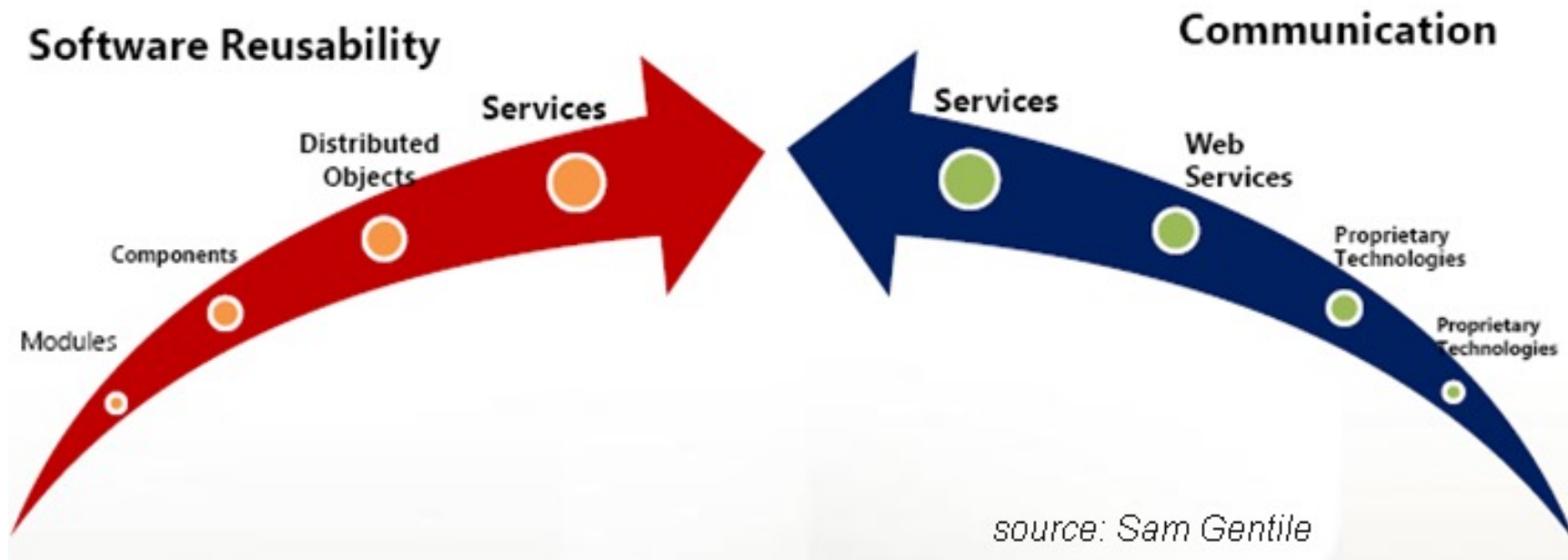
Các thành phần Kiến trúc hướng dịch vụ (2)

- ❖ **Nguyên tắc phân chia (Separation of Concerns):** SOA tách riêng các loại nghiệp vụ khác nhau thành các dịch vụ riêng biệt, giúp tăng tính tái sử dụng và linh hoạt.
- ❖ **Tính Tích hợp (Interoperability):** SOA hỗ trợ tích hợp các ứng dụng và dịch vụ từ nhiều nguồn khác nhau, không phụ thuộc vào các công nghệ cụ thể hoặc hãng cụ thể.
- ❖ **Tính Phản ứng (Loose Coupling):** Các dịch vụ trong SOA được thiết kế sao cho chúng không phụ thuộc mạnh mẽ vào nhau, giúp dễ dàng thay đổi, mở rộng và tái sử dụng.
- ❖ **Tính Tính toán phân tán (Distributed Computing):** SOA cho phép triển khai các dịch vụ trên nhiều máy chủ và nền tảng khác nhau, tạo ra một môi trường phân tán.

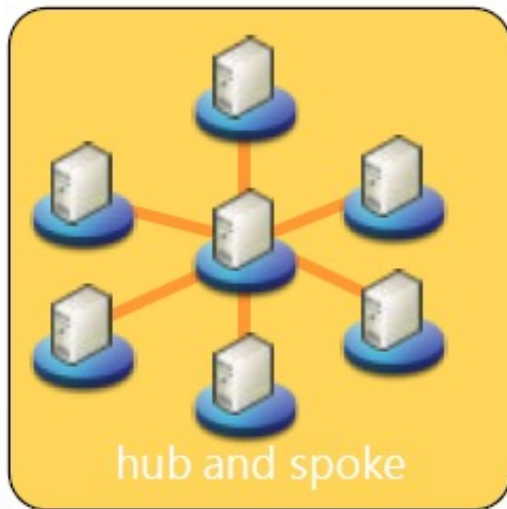
Kiến trúc phân tầng hệ thống SOA



SOA là hướng tiếp cận tiên bộ

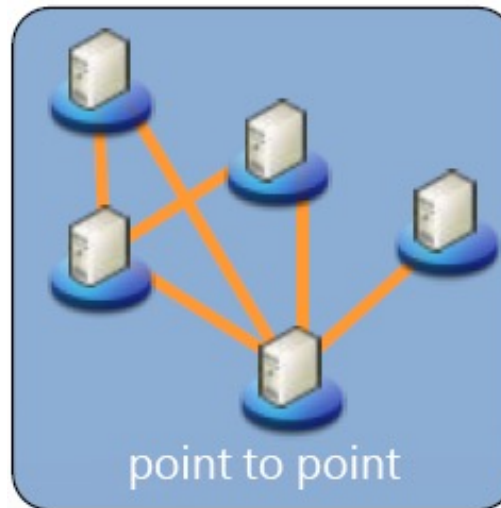


SOA là hướng tiếp cận tiến bộ



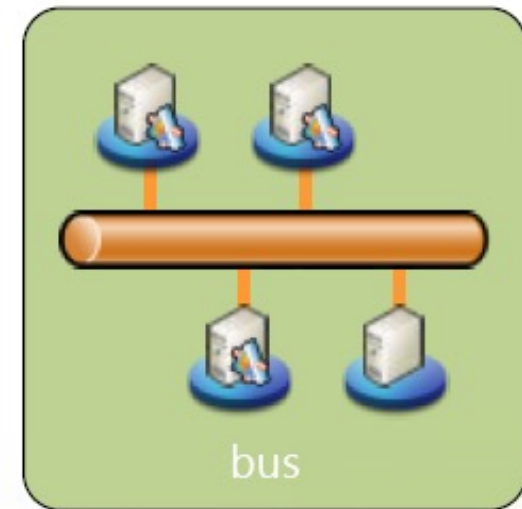
"too centralized"

EAI



"too decentralized"

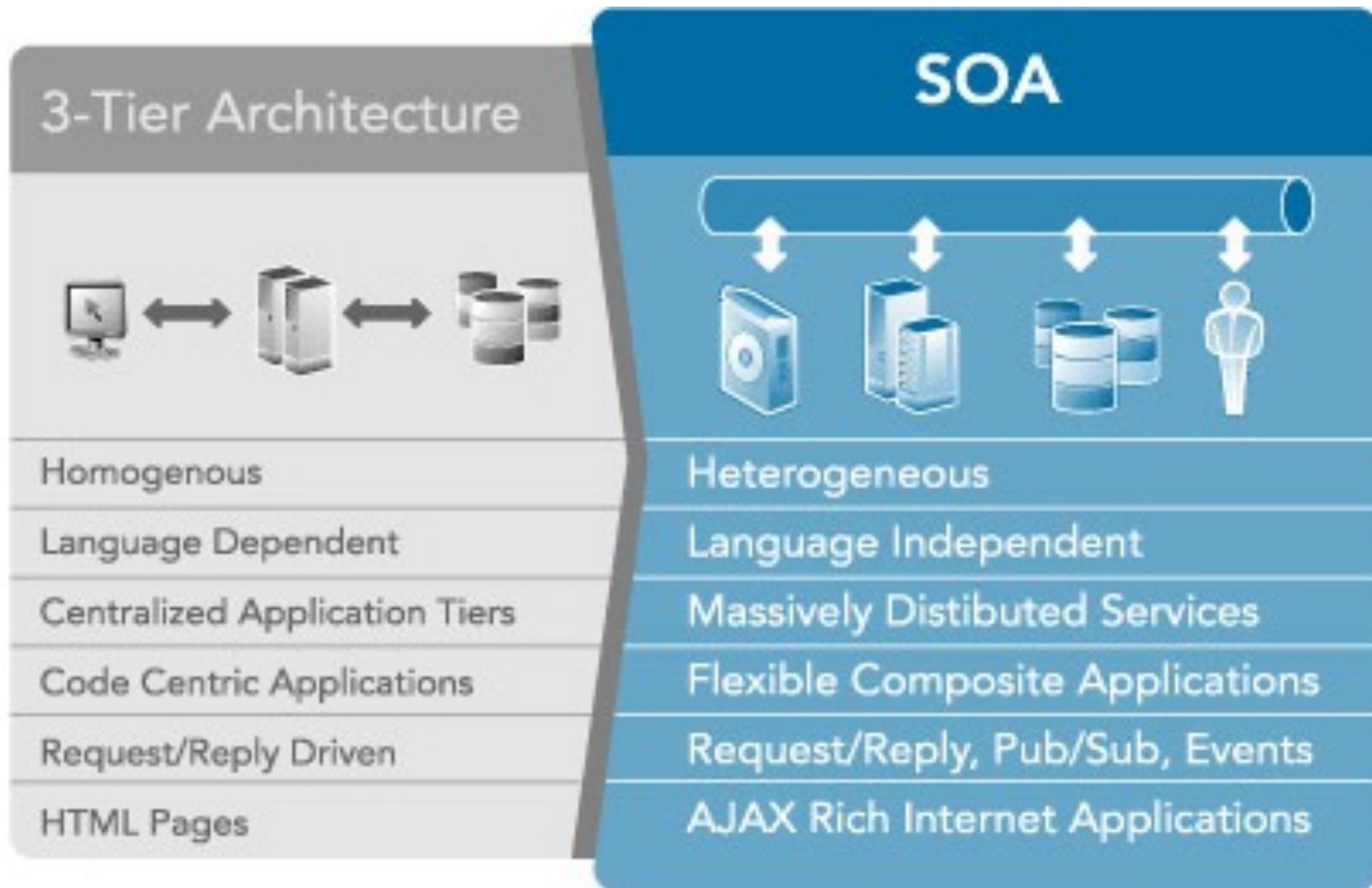
Project-ware



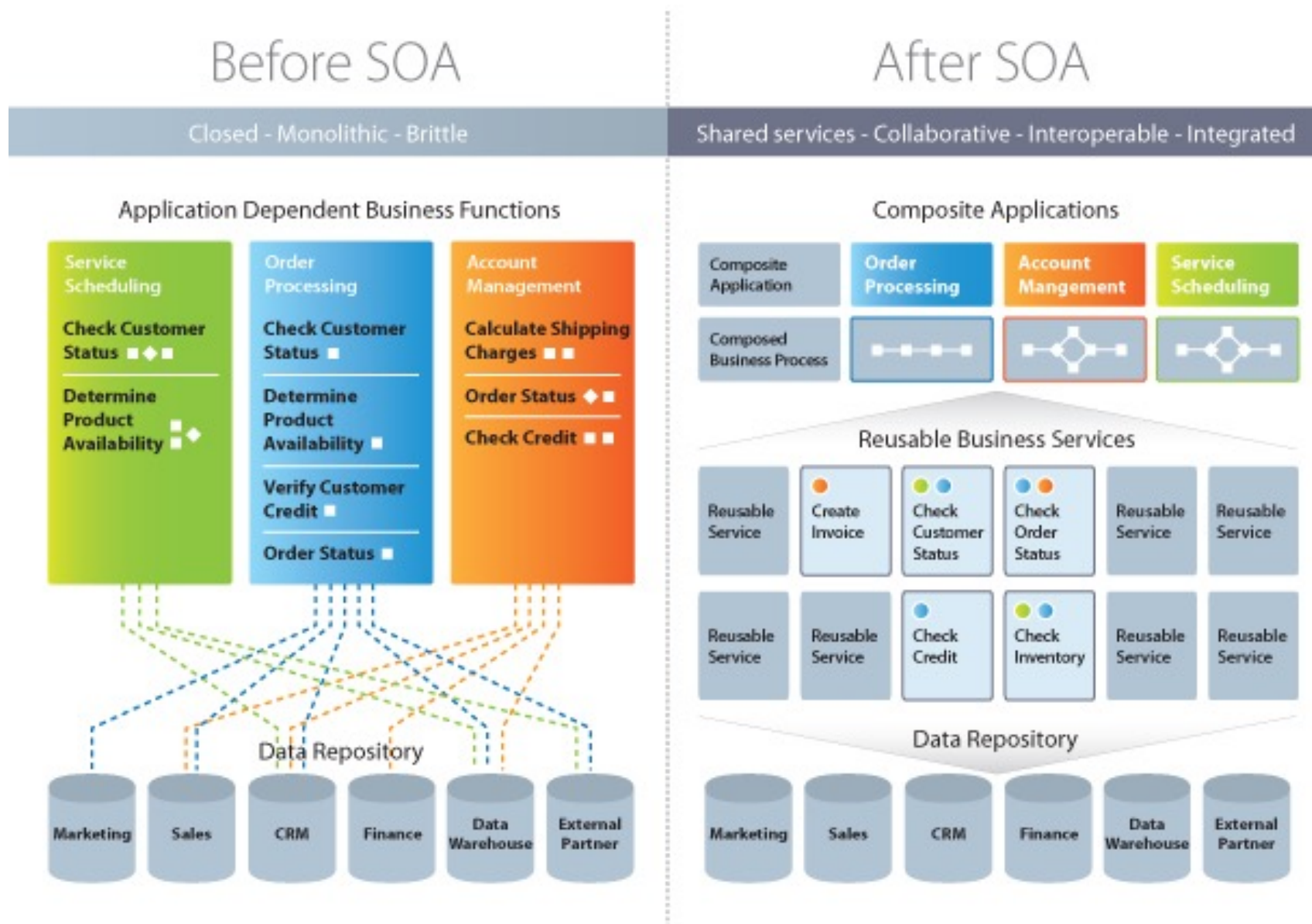
"just right"

SOA

SOA là hướng tiếp cận tiến bộ



SOA là hướng tiếp cận tiên bộ



Nguyên tắc xây dựng kiến trúc hướng dịch vụ

❖ Sự phân định ranh giới rạch ròi giữa các dịch vụ:

- Các dịch vụ thực hiện quá trình tương tác chủ yếu thông qua thành phần giao tiếp.
- Thành phần giao tiếp qui định về những định dạng thông điệp sử dụng trong quá trình trao đổi - cách duy nhất để các đối tượng bên ngoài có thể truy cập thông tin và chức năng của dịch vụ.
- Ta chỉ cần gửi các thông điệp theo các định dạng đã được định nghĩa trước mà không cần phải quan tâm đến cách xử lý của dịch vụ như thế nào

Nguyên tắc xây dựng kiến trúc hướng dịch vụ

❖ Các dịch vụ tự hoạt động:

- Các dịch vụ cần phải được triển khai và hoạt động như những thực thể độc lập mà không lệ thuộc vào một dịch vụ khác.
- Dịch vụ phải có tính bền vững cao.
- để tránh các cuộc tấn công từ bên ngoài (như gửi thông điệp lỗi, hay gửi thông điệp ồ ạt) bằng cách sử dụng các kỹ thuật về an toàn, bảo mật

Nguyên tắc xây dựng kiến trúc hướng dịch vụ

❖ Các dịch vụ chia sẻ lược đồ

- Các dịch vụ nên cung cấp thành phần giao tiếp của nó (interface) ra bên ngoài, hỗ trợ chia sẻ các cấu trúc thông tin, các ràng buộc dữ liệu thông qua các lược đồ dữ liệu (schema) chuẩn (độc lập ngôn ngữ, độc lập hệ nền.).
- Hệ thống sẽ có tính liên kết và khả năng dễ mở rộng.

Nguyên tắc xây dựng kiến trúc hướng dịch vụ

❖ Tính tương thích của dịch vụ dựa trên chính sách

- Một dịch vụ khi muốn tương tác với một dịch vụ khác thì phải thỏa mãn các chính sách (policies) và yêu cầu (requirements) của dịch vụ đó như là mã hóa, bảo mật, vv.

→ Để thực hiện điều này, mỗi dịch vụ cần phải cung cấp công khai các yêu cầu, chính sách đó

Ưu điểm của SOA

- ❖ Sử dụng lại những thành phần hoặc dịch vụ có sẵn giúp tiết kiệm chi phí và thời gian xây dựng ứng dụng
- ❖ Cung cấp giải pháp ứng dụng tổng hợp cho doanh nghiệp (tích hợp các ứng dụng rời rạc thành hệ thống đồng nhất)
- ❖ Tính loose coupling (kết nối lỏng lẻo) giúp tăng tính linh hoạt và khả năng triển khai cài đặt
- ❖ Thích ứng với những thay đổi trong tương lai (giảm rủi ro do có thay đổi hoặc lỗi trong thiết kế hệ thống)
- ❖ Hỗ trợ đa thiết bị và đa nền tảng
- ❖ Tăng khả năng mở rộng và khả năng sẵn sàng cung cấp

Ưu điểm của SOA

❖ Loose coupling

- các module có tính loose coupling khi có một số ràng buộc được mô tả rõ ràng
- Hầu như mọi kiến trúc phần mềm đều hướng đến tính loose coupling giữa các module.
- Mức độ kết dính của mỗi hệ thống ảnh hưởng trực tiếp đến khả năng chỉnh sửa hệ thống của chính nó

Ưu điểm của SOA

❖ Sử dụng lại dịch vụ

- Bởi vì các dịch vụ được cung cấp lên trên mạng và được đăng ký ở một nơi nhất định (service registry) nên chúng dễ dàng được tìm thấy và tái sử dụng
- Tái sử dụng lại các dịch vụ giúp loại bỏ những thành phần trùng lặp, tăng độ vững chắc trong cài đặt, đơn giản hoá việc quản trị
- Những dịch vụ được dùng chung bởi tất cả các ứng dụng của một hệ thống SOA gọi là những shared infrastructure service

Ưu điểm của SOA

❖ Khả năng cộng tác

- các hệ thống có thể giao tiếp với nhau trên nhiều nền tảng và ngôn ngữ khác nhau.
- Mỗi dịch vụ cung cấp một interface có thể được triệu gọi thông qua một dạng kết nối.
- Một kết nối gọi là interoperable chứa bên trong nó một giao thức và một định dạng dữ liệu mà mỗi client kết nối đến nó đều hiểu

Ưu điểm của SOA

❖ Tự phục hồi

- Là một hệ thống có khả năng tự hồi phục sau khi bị lỗi mà không cần sự can thiệp của con người.
- Độ tin cậy (reliability) là mức độ đo khả năng một hệ thống xử lý tốt như thế nào trong tình trạng hỗn loạn; phụ thuộc vào khả năng phục hồi của phần cứng sau khi bị lỗi
- Nếu một thể hiện service nào đó không hoạt động thì một thể hiện khác vẫn có thể hoàn tất giao dịch cho khách hàng mà không bị ảnh hưởng gì
- → Đây là một trong những tính chất cơ bản của các hệ thống hướng dịch vụ.

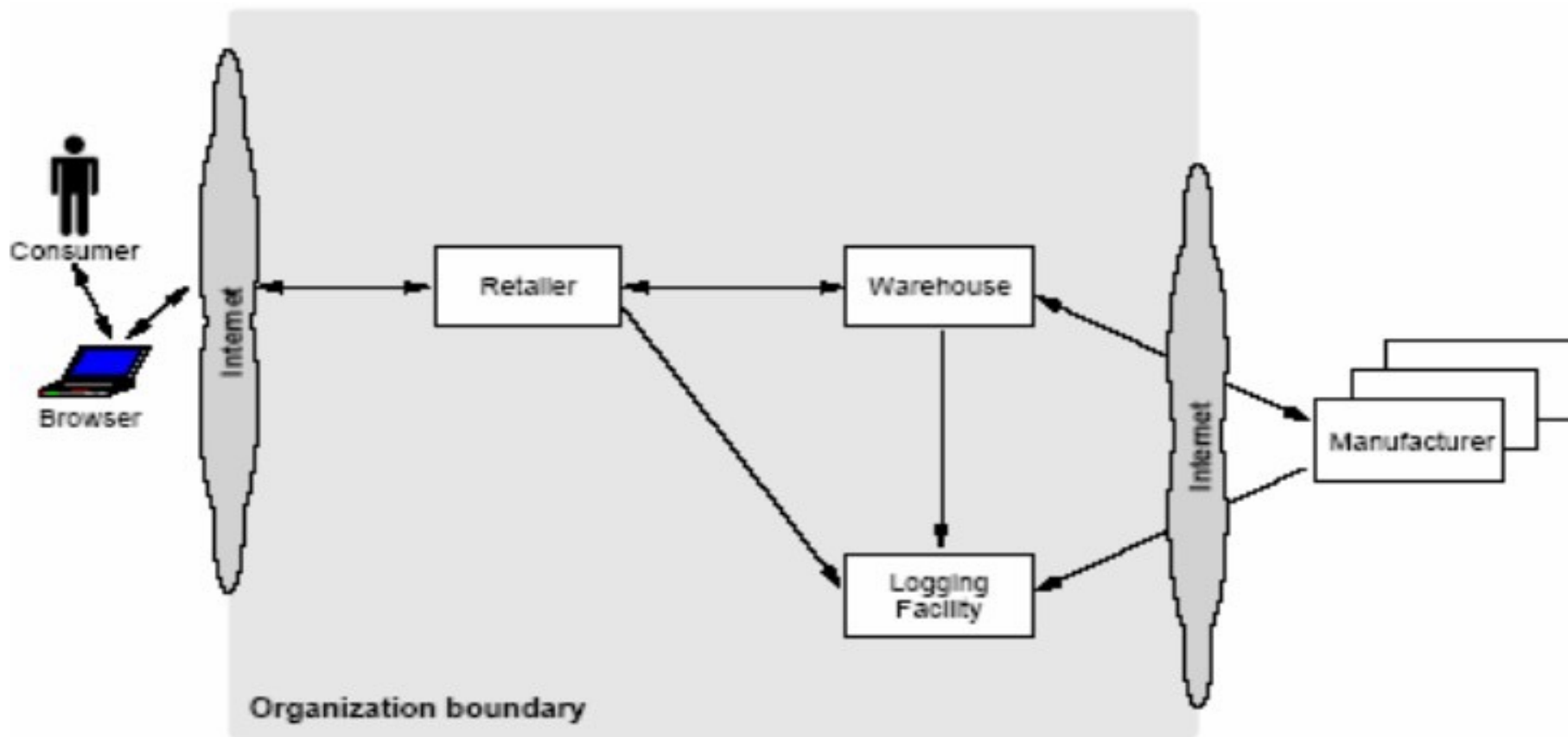
Nhược điểm SOA

- ❖ Khó khăn trong xác định, phân tích và thiết kế dịch vụ
- ❖ Khó khăn trong tích hợp các dịch vụ
- ❖ Khó miêu tả dữ liệu không cấu trúc trong header của message.
- ❖ Vấn đề bảo mật do các dịch vụ có thể đa nền tảng và quản lý bởi các chủ sở hữu khác nhau. Khi xây dựng ứng dụng tổng hợp từ nhiều dịch vụ với tính tái sử dụng cao thì vấn đề bảo mật như: xác thực, phân quyền, bí mật và an toàn vẹn dữ liệu, bảo vệ quyền riêng tư... trở thành một bài toán phức tạp và đòi hỏi giải quyết bằng những hướng tiếp cận bảo mật hoàn toàn mới so với các phương pháp bảo mật truyền thống.
- ❖ Băng thông mạng phải đảm bảo, do các dịch vụ giao tiếp và triệu gọi nhau thông qua hình thức gửi thông điệp XML

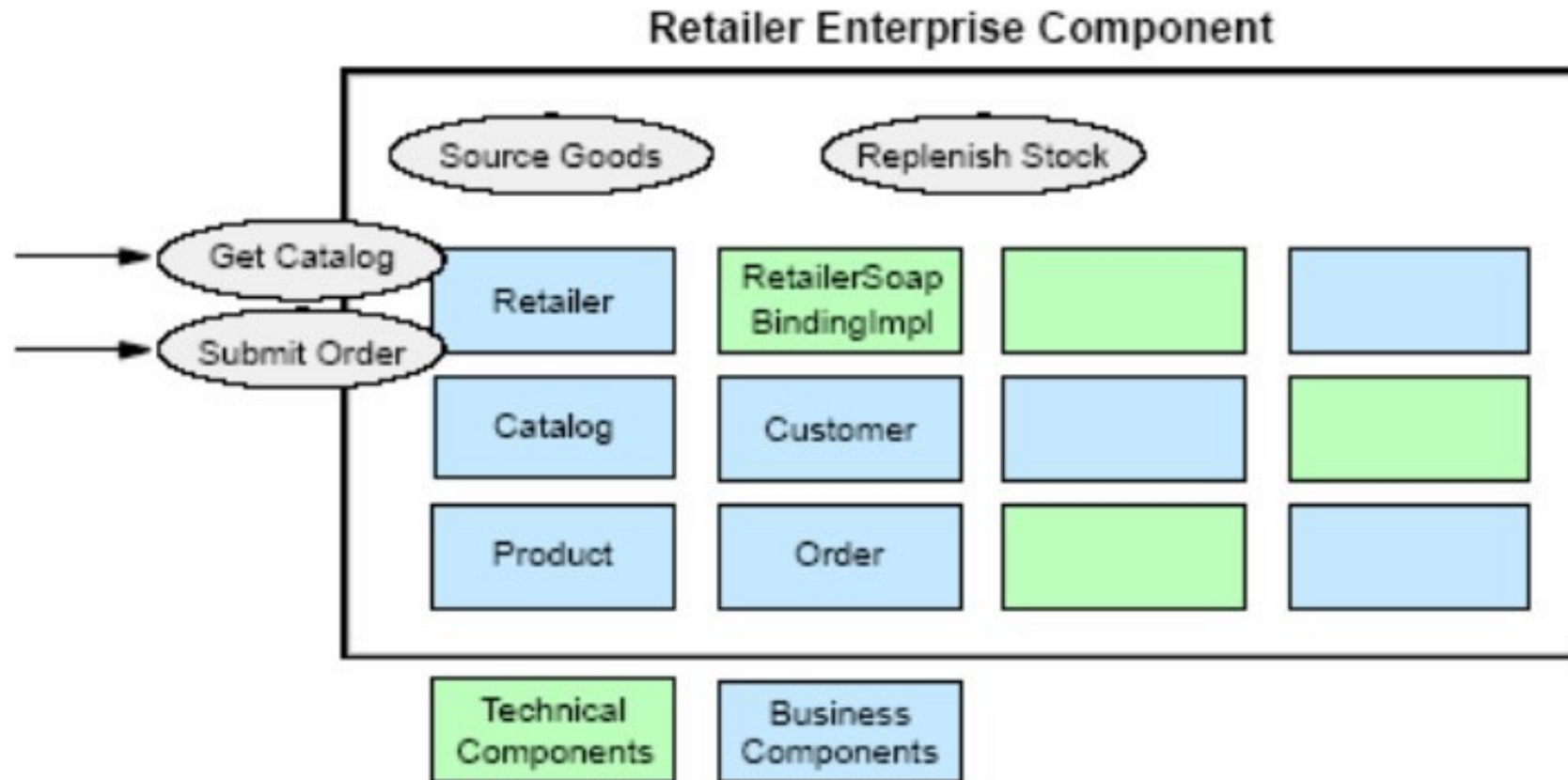
Ví dụ thực tế

- ❖ Khách hàng (customer) sẽ truy cập vào website của đại lý (retailer) để xem qua các mặt hàng và đặt hàng các sản phẩm điện tử như là TV, đầu DVD và video camera.
- ❖ Đại lý chuyển yêu cầu (order) đến cho bộ phận thủ kho (warehouse) để xử lý.
- ❖ Nếu nguồn hàng trong kho dưới mức cho phép thì yêu cầu bổ sung hàng (replenishment order) được gửi đến nhà sản xuất (manufacturer).
- ❖ Nhà sản xuất không giải quyết ngay yêu cầu bổ sung hàng, mà có thể sau một khoảng thời gian nào đó khi sản xuất đủ lượng hàng.

Ví dụ thực tế



Ví dụ thực tế



Kiến trúc hướng sự kiện (Event-Driven Architecture - EDA)

Sự kiện là gì?

- ❖ Sự kiện được định nghĩa là sự thay đổi trạng thái của một số hệ thống kinh doanh chính. Bất cứ khi nào điều gì đó xảy ra bên trong hoặc bên ngoài tổ chức của bạn, khi đó một sự kiện đã xảy ra.
- ❖ Những hành động này có thể là bất cứ điều gì: khách hàng mua hàng, cập nhật hàng tồn kho trong kho, cố gắng vi phạm bảo mật, thay đổi trạng thái của ứng dụng, lượt truy cập trang web, bỏ qua giỏ hàng...
- ❖ Trong thời đại kỹ thuật số, các sự kiện được xử lý bởi các hệ thống CNTT ghi lại, xử lý và phản hồi các sự kiện dựa trên logic đặt trước.

Sự kiện trong thiết kế hệ thống

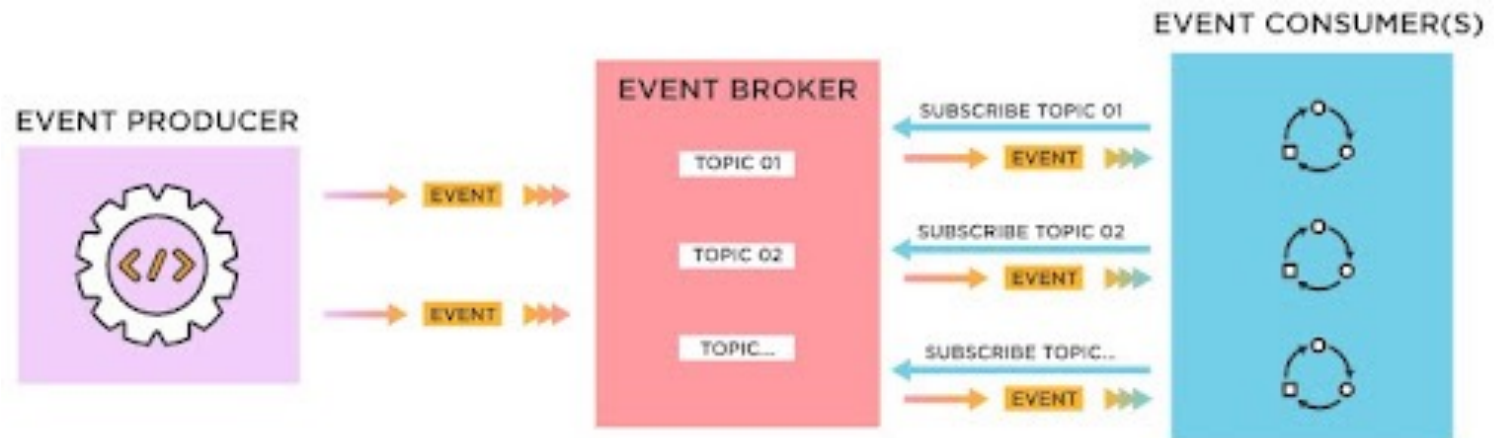
- ❖ Chúng là một bản record rằng điều gì đó đã xảy ra.
- ❖ Chúng là bất biến - chúng không thể thay đổi hoặc xóa được.
- ❖ Chúng có thể tồn tại vô thời hạn - các sự kiện có thể được lưu trữ và truy cập mãi mãi.
- ❖ Chúng có thể được sử dụng không giới hạn số lần - không giới hạn số lần một sự kiện có thể được xử lý bởi một dịch vụ.

Kiến trúc hướng sự kiện

- ❖ Kiến trúc hướng sự kiện xử lý thông báo sự kiện - thông báo sự kiện là một tin nhắn xác nhận một hành động đã được thực hiện. Thông báo có thể là một xác nhận đơn giản về hành động hoặc chứa dữ liệu bổ sung về sự kiện mà các hệ thống hạ lưu sử dụng để áp dụng logic nghiệp vụ của chúng.
- ❖ Ví dụ: thông báo mua hàng có thể bao gồm ID của khách hàng, ID sản phẩm, tổng số lần mua, ngày và giờ mua và các chỉ số khác mà CRM sử dụng để tạo hồ sơ khách hàng và dự đoán các sản phẩm trong tương lai mà họ có thể cần.

Kiến trúc hướng sự kiện

- ❖ Kiến trúc hướng sự kiện (Event-Driven Architecture - EDA) là một phương pháp thiết kế hệ thống được xây dựng để ghi lại, truyền tải và xử lý các sự kiện thông qua một kiến trúc tách rời.
- ❖ Điều này có nghĩa là các hệ thống không cần biết về nhau để chia sẻ thông tin và hoàn thành nhiệm vụ. Kiến trúc này thay thế kiến trúc “request/response” truyền thống, nơi các dịch vụ sẽ phải đợi phản hồi trước khi chúng có thể chuyển sang tác vụ tiếp theo. Luồng của kiến trúc hướng sự kiện được điều hành bởi các sự kiện và nó được thiết kế để phản hồi chúng hoặc thực hiện một số hành động để phản hồi lại một sự kiện.



Kiến trúc hướng sự kiện

- ❖ Các tổ chức tận dụng EDA khi thiết kế hệ thống để đạt được nhiều trường hợp sử dụng khác nhau như:
 - **Sao chép dữ liệu:** Một sự kiện có thể được chia sẻ giữa nhiều dịch vụ cần sao chép dữ liệu của nó vào cơ sở dữ liệu của chúng.
 - **Xử lý song song:** Nhiều quá trình có thể được kích hoạt bởi một sự kiện để thực thi không đồng bộ của nhau.
 - **Giám sát thời gian thực:** Các hệ thống có thể tạo ra các sự kiện cho các thay đổi đối với trạng thái của chúng để một tổ chức có thể quét các điểm bất thường và các hoạt động đáng ngờ.
 - **Khả năng tương tác:** Các sự kiện có thể được tồn tại và lan truyền bất kể dịch vụ mã được viết bằng gì.
 - **Dự phòng:** Nếu một dịch vụ không hoạt động, các sự kiện có thể được duy trì trong bộ định tuyến cho đến khi dịch vụ khả dụng để sử dụng sự kiện.
 - **Microservices:** EDA thường được ghép nối với microservices để chia sẻ thông tin hiệu quả giữa các hệ thống được tách rời trên quy mô lớn.

Ưu điểm của Kiến trúc hướng sự kiện

- ❖ **Khả năng chịu lỗi cao:** tính năng tách rời mang lại cho EDA khả năng chịu lỗi cao vì lỗi trong một dịch vụ sẽ không ảnh hưởng đến các dịch vụ còn lại.
- ❖ **Lưu vào bộ đệm:** do tính chất không đồng bộ của EDA, không cần đợi consumer bắt kịp nếu các sự kiện được tạo với tốc độ khác với tốc độ chúng được tiêu thụ.
- ❖ **Khả năng mở rộng:** mỗi dịch vụ có thể được mở rộng một cách độc lập để phù hợp nhất với nhu cầu của doanh nghiệp.

Ưu điểm của Kiến trúc hướng sự kiện

- ❖ **Tiết kiệm chi phí:** Việc chuyển đổi các quy trình nghiệp vụ thành kiến trúc hướng sự kiện loại bỏ nhu cầu về các quy trình hàng loạt này và thay vào đó cho phép một luồng dữ liệu liên tục được xử lý khi nó được tạo ra. Cách tiếp cận xử lý liên tục này cho phép doanh nghiệp đạt được thời gian phản hồi có thể dự đoán được và khả năng mở rộng xử lý, do đó giảm số lượng cơ sở hạ tầng cần thiết để xử lý định kỳ một lượng lớn dữ liệu. Do đó, điều này làm giảm chi phí hoạt động của việc trích xuất giá trị từ dữ liệu và tăng hiệu quả tổng thể, năng suất và lợi nhuận. EDA cũng tiêu thụ ít băng thông hơn.
- ❖ **Nhanh nhẹn:** Các sự kiện được tự động lọc và định tuyến mà không cần tạo mã tùy chỉnh dài dòng.

Ưu điểm của Kiến trúc hướng sự kiện

- ❖ **Vị trí tập trung:** Luồng sự kiện hoạt động như một vị trí tập trung, nơi các thay đổi chính sách và kiểm tra có thể được thực hiện.
- ❖ **Nâng cao trải nghiệm khách hàng:** EDA là một phương pháp tiếp cận năng động đối với trải nghiệm khách hàng và quản lý khách hàng - một phương pháp giúp dễ dàng đáp ứng kỳ vọng của khách hàng xung quanh việc cá nhân hóa mà không bỏ lỡ bất kỳ khoảnh khắc tương tác nào.

Ưu điểm của Kiến trúc hướng sự kiện

- ❖ Với việc EDA ngày càng trở thành một yếu tố thiết yếu của việc triển khai microservice cũng như các chiến lược kinh doanh và kỹ thuật lớn hơn, điều quan trọng là phải chọn các giải pháp API cho phép bạn dễ dàng quản lý, bảo mật và chi phối các API đồng bộ và theo hướng sự kiện của mình.
- ❖ EDA là một cách tiếp cận mạnh mẽ để thiết kế hệ thống mang lại hiệu quả và tính linh hoạt cho các tổ chức muốn giải quyết nhanh chóng các sự kiện quan trọng đối với doanh nghiệp của họ. Nó cho phép thực hiện song song, sao chép dữ liệu, khả năng tương tác và các trường hợp sử dụng khác một cách dễ dàng.
- ❖ EDA là một phương pháp trung tâm để đạt được một tổ chức theo hướng sự kiện, nơi các hành động kích hoạt phản ứng ngay lập tức, tăng sự nhanh nhẹn và mở rộng quy mô để đáp ứng nhu cầu của khách hàng.

HỎI/ĐÁP

